Domain modelling Component diagram

Katerina Mangaroska week 36

01

4+1 views paradigm

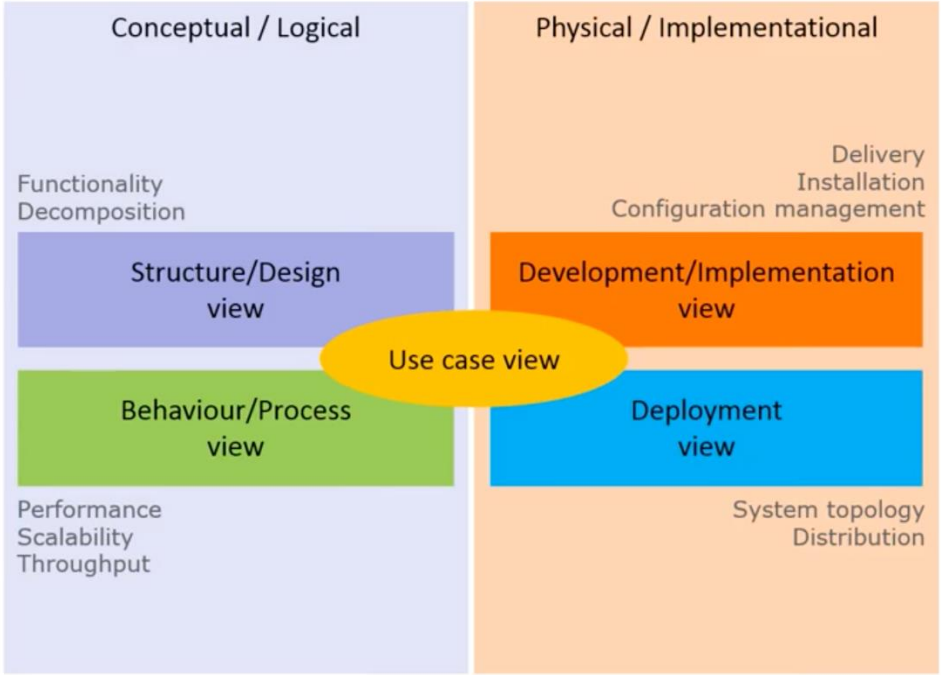
Component diagram
Class diagram

Sequence diagram
State diagram
Activity diagram

Mapping Components onto files

Deployment diagram

Use case diagram
Context diagram

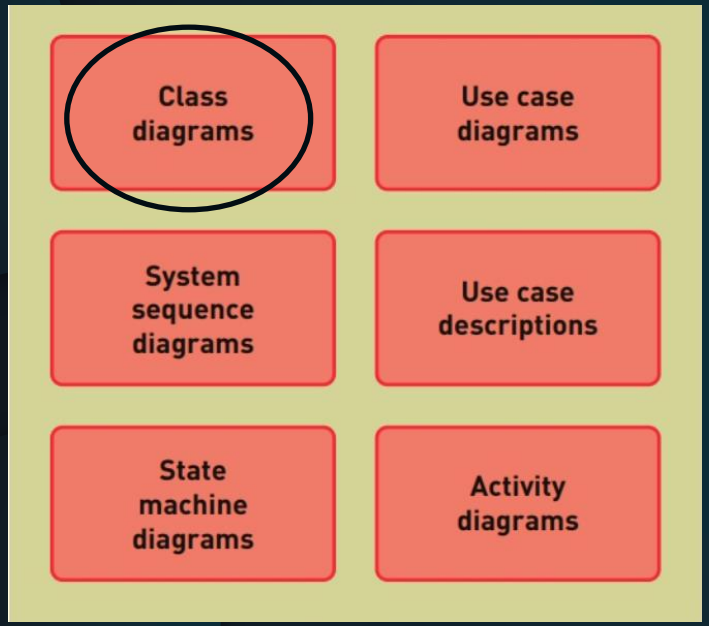


*Kruchten '95

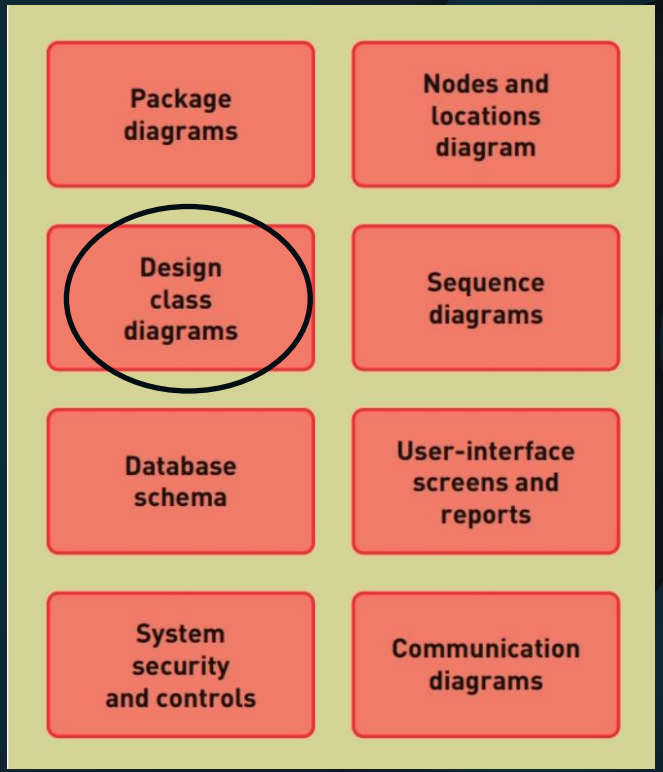
Structure design view is concerned with the **functionality that the system provides to end-users**. UML diagrams that are used to present this view are component diagram and class diagram. This is also known as logical view.

Class is the main building block of any object-oriented solution.

analysis models



design models



02

The problem domain

Problem domain

...consists of those functional areas of the business that are affected by the problem in question or that may be impacted by a solution to that problem. It is the specific area of the user's business that is included within the scope of the new system.

Problem space - all information that defines the problem and constrains the solution (the constraints being part of the problem).

Why defining it is important? -- to clearly understand where the problem lives in the business processes. To clearly see the conditions that cause the problem and to see what other business processes may be impacted by any solution considered.

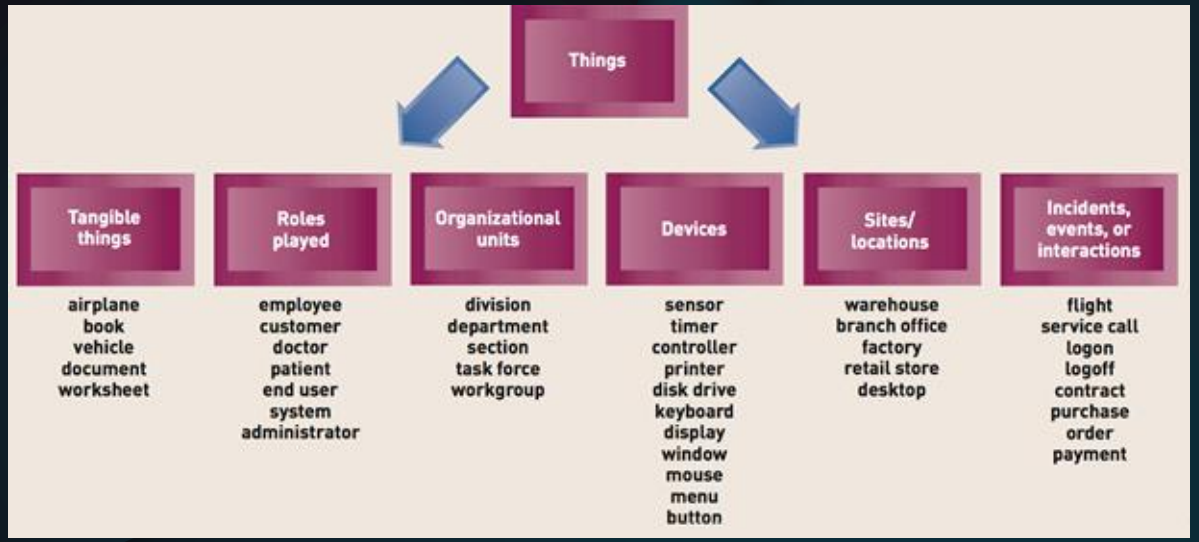
Problem domain

A domain model captures the most important types of objects in the context of the business. The domain model represents the ‘things’ that exist or events that transpire in the business environment.

Domain classes are what end users deal with when they work (customers, shipments, products).

OOA defines the objects that do the work and determines what user interactions (use cases) are required to complete the tasks.

- An object is a thing in an IS, capable of responding to messages by executing functions.
- Two techniques - the brainstorming technique and the noun technique to identify things or objects.



The brainstorming technique

What type of things are affected that the system needs to know about and store information about?

What are things? – things are related to the people who interact with the system or to other stakeholders. e.g., *a customer is a person who places an order*, but the system needs to store information about that customer, so a *customer is also a thing* in the problem domain. Things can fit into more than one type; e.g., vehicle is a tangible thing and a device.

Incidents can be considered as **associations** between things; e.g., *sale* is an association between a customer and an item of inventory.



The brainstorming technique

Example: **A customer wants to buy a product online.**
What are the things you can define important for the system to know and store, presented in a use case?

- tangible things: item/s ordered
- roles played: the customer
- organizational units: shipping, inventory
- locations: warehouse
- incidents: the sale

Steps to follow for the brainstorming technique

1. Identify a user and a set of use cases.
2. Brainstorm with the user to identify things involved when carrying out the use case, that is, things about which information should be captured by the system.
3. Use the types of things (categories) to systematically ask questions about potential things, such as the following: Are there any tangible things you store information about? Are there any locations involved? Are there roles played by people that you need to remember?
4. Continue to work with all types of users and stakeholders to expand the brainstorming list.
5. Merge the results, eliminate any duplicates, and compile an initial list.

The noun technique

Attributes are descriptive pieces of information about things or objects.

Systems store and use specific pieces of information about each thing.

How it differs? - You lists all nouns without thinking too much about them and without talking much to users. e.g., a customer has a name, a phone number, an address, a credit info…

Identifier or key is an attribute that uniquely identifies a thing.

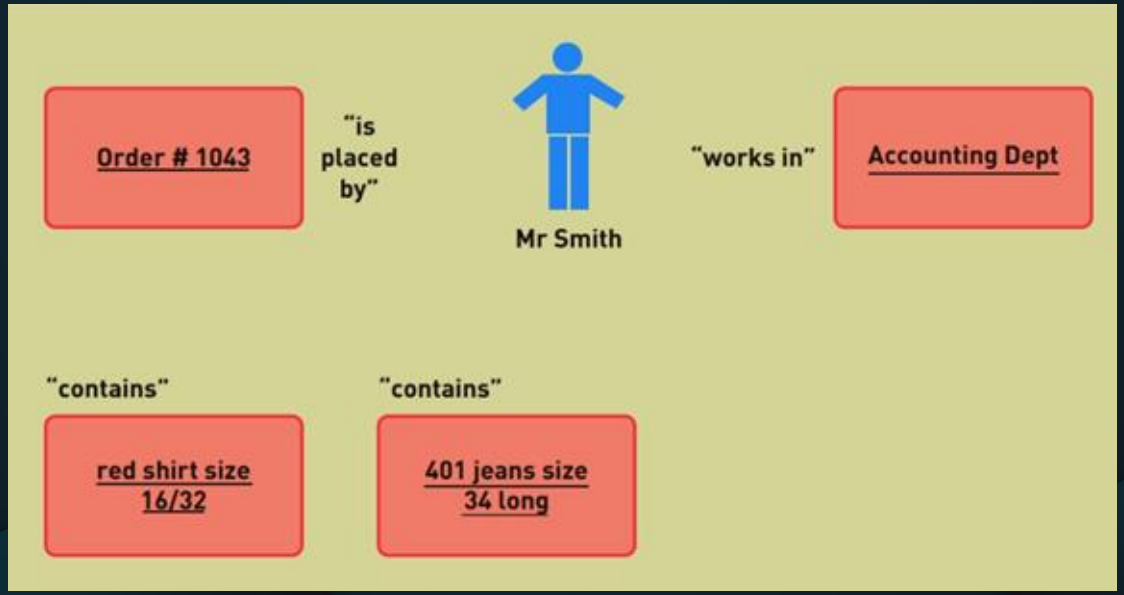
e.g., a Social Security number, vehicle ID number, product ID number, invoice number, or transaction number assigned by the system.

A compound attribute is an attribute that contains a collection of related attributes.

e.g., Customer full name - a first name, a middle name, a last name, and a nickname.
e.g., Customer telephone number – home number, work number, mobile phone

Associations among things

Association is a term, in UML, that describes a naturally occurring relationship between specific things, sometimes called a relationship. Information systems need to store information about the specific associations.



Associations among things

Associations apply in two directions.

- e.g., a customer places an order (association in one direction).
- e.g., an order is placed by a customer (the association in the other direction).

What does the system need to store?

The nature of each associations in terms of links:

CARDINALITY is a measure of the *number of links* between one object and another object in a relationship. (database management)

MULTIPLICITY is a measure, in UML, of the *number of links* between one object and another object in an association.

*multiplicity is equal to cardinality in UML

The min and max multiplicity

The range of possible values of the multiplicity.

e.g., a customer might not ever place an order --> zero associations

a customer might place one order --> one association exists

a customer might place two, three, or more orders --> many associations

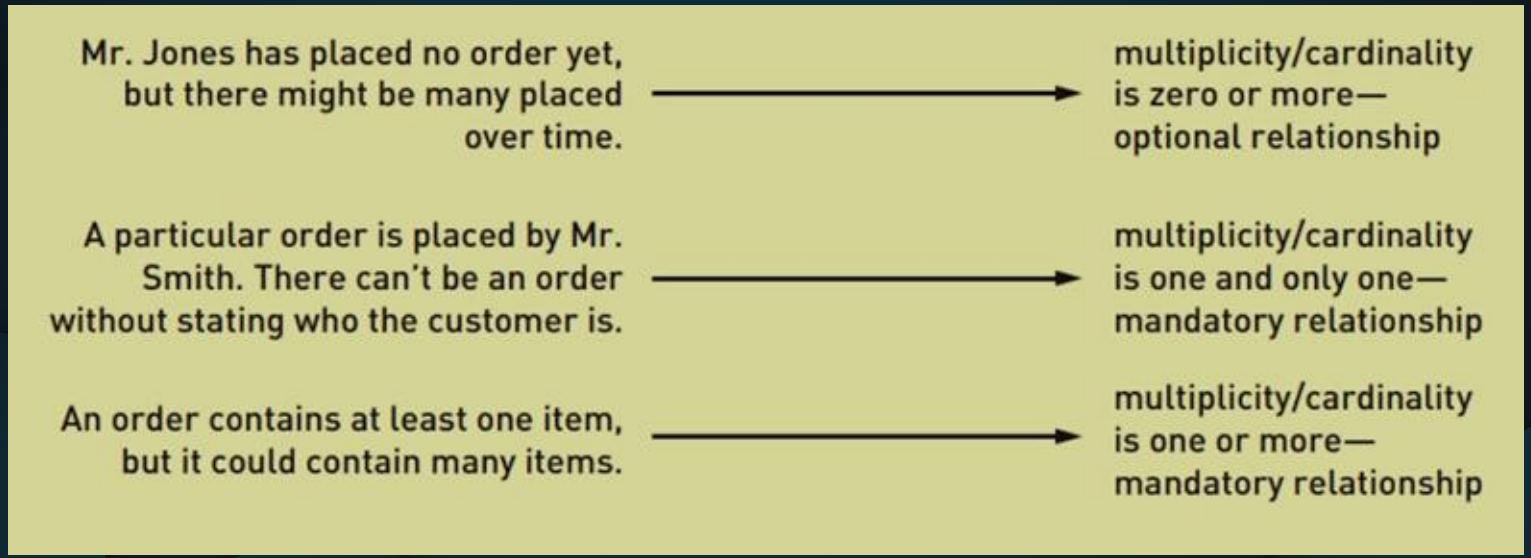
The relationship for a customer placing an order can have a range of zero, one, or more, usually indicated as **zero or more**. The zero is the min multiplicity, and more is the max multiplicity.

a mandatory vs an optional association

- the system might not record any information about a customer until the customer places an order.

Multiplicity constraints

Multiplicity constraints is the actual numeric count of the constraints on objects allowed in an association.



Types of associations

- Binary association is an association between exactly two distinct types of things.
  - e.g., a customer and an order
- Unary association is an association between two instances of the same type of thing.
  - e.g., the association is married to, which is between two people.
  - e.g., an organizational hierarchy – the packing department reports to shipping, which reports to distribution, which reports to marketing.
- Ternary association is an association between exactly three distinct types of things.
  - e.g., a customer, an order, and a sales representative

n-ary association is an association between n distinct types of things.

03

Class diagram

The Domain model

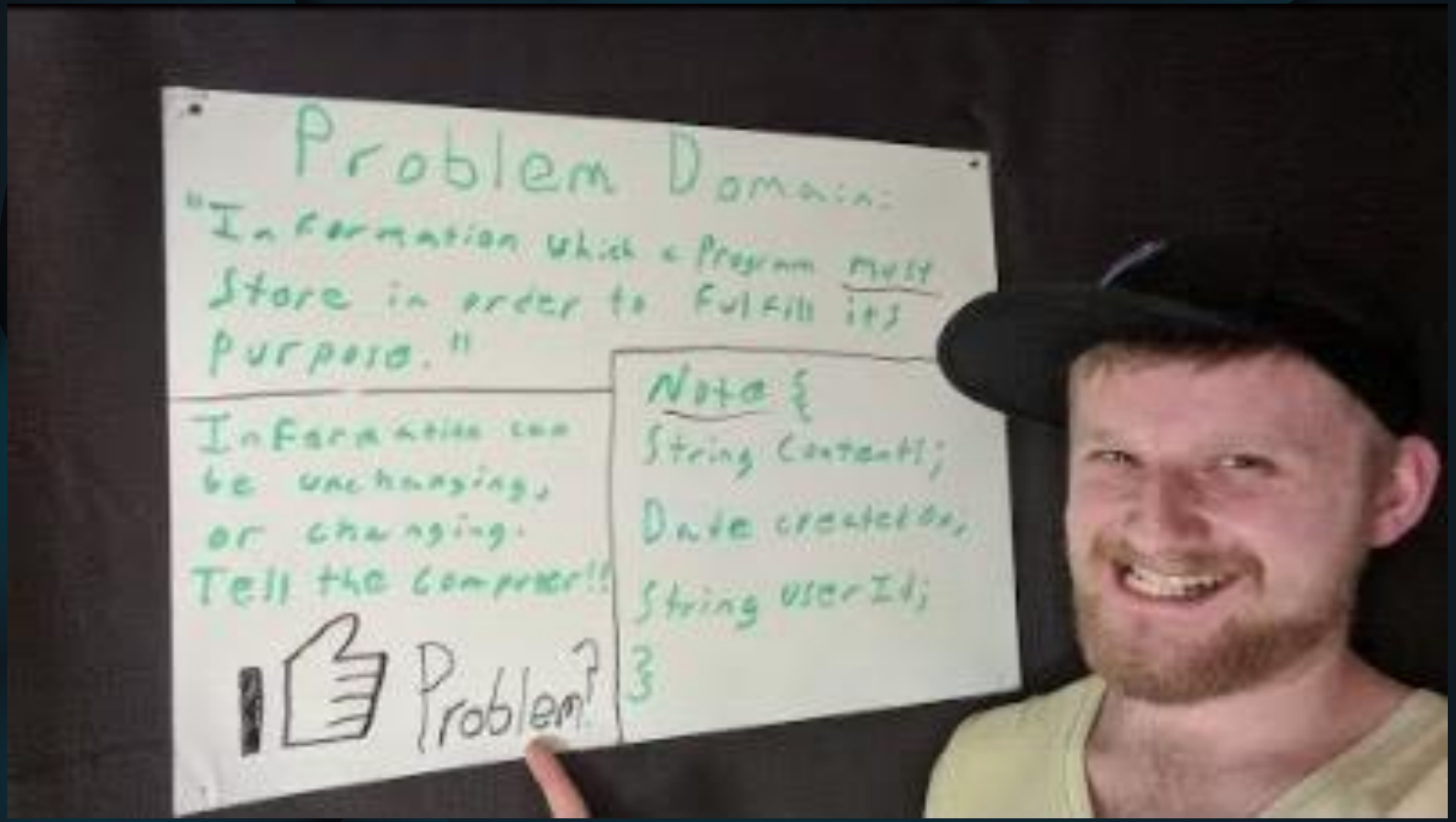
Domain Modeling

In software engineering – Domain model is a conceptual model of the domain that incorporates behavior and data.

In ontology engineering – Domain model is a formal representation of a knowledge domain with concepts, roles, data types, individuals, and rules, grounded in a description logic.

Domain models are developed as descriptions of a problem domain that are not dependent on any solution.

Visual representation of conceptual classes.
Does not include operations / functions.
Does not describe software classes.
Does not describe software responsibilities.


Problem Domain:
"Information which a Program must Store in order to Fulfill its purpose."
Information can be unchanging, or changing.
Tell the Computer!!
Problem?
Note {
String Content;
Date creationDate;
String userId;
}

Domain Modeling

The Domain Model is your organized and structured knowledge of the problem. The Domain Model should represent the vocabulary and key concepts of the problem domain and it should identify the relationships among all the entities within the scope of the domain.

A domain model is a visual representation of conceptual classes or real-world objects in a domain of interest.


STATISTICS
VS
DOMAIN
EXPERTISE
WITH GOOGLE'S
CASSIE KOZYRKOV

The domain model class diagram

Domain classes are classes that describe things in the problem domain.

A **domain model class diagram** is a class diagram that only includes classes from the problem domain.

Why? - To model the things in the problem domain.

The design class diagram is a refinement of the domain model class diagram and is used to represent software classes in the new system.

Domain Class Diagrams are conceptual, while Design Class Diagrams are logical.

The domain model

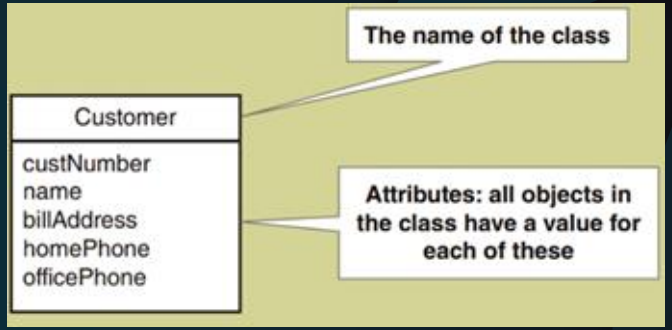
- **Rectangles** represent classes, and the **lines connecting** the rectangles show the associations among classes.
- The **domain class** symbol is a rectangle with two sections.

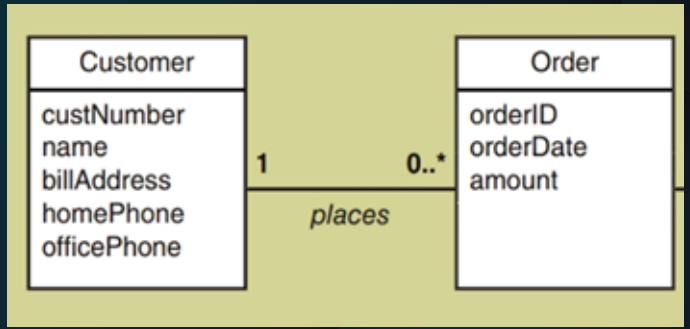
*methods are not applied to problem domain classes.

- Class names and attribute names use camelback notation, in which the words run together without a space or underscore.

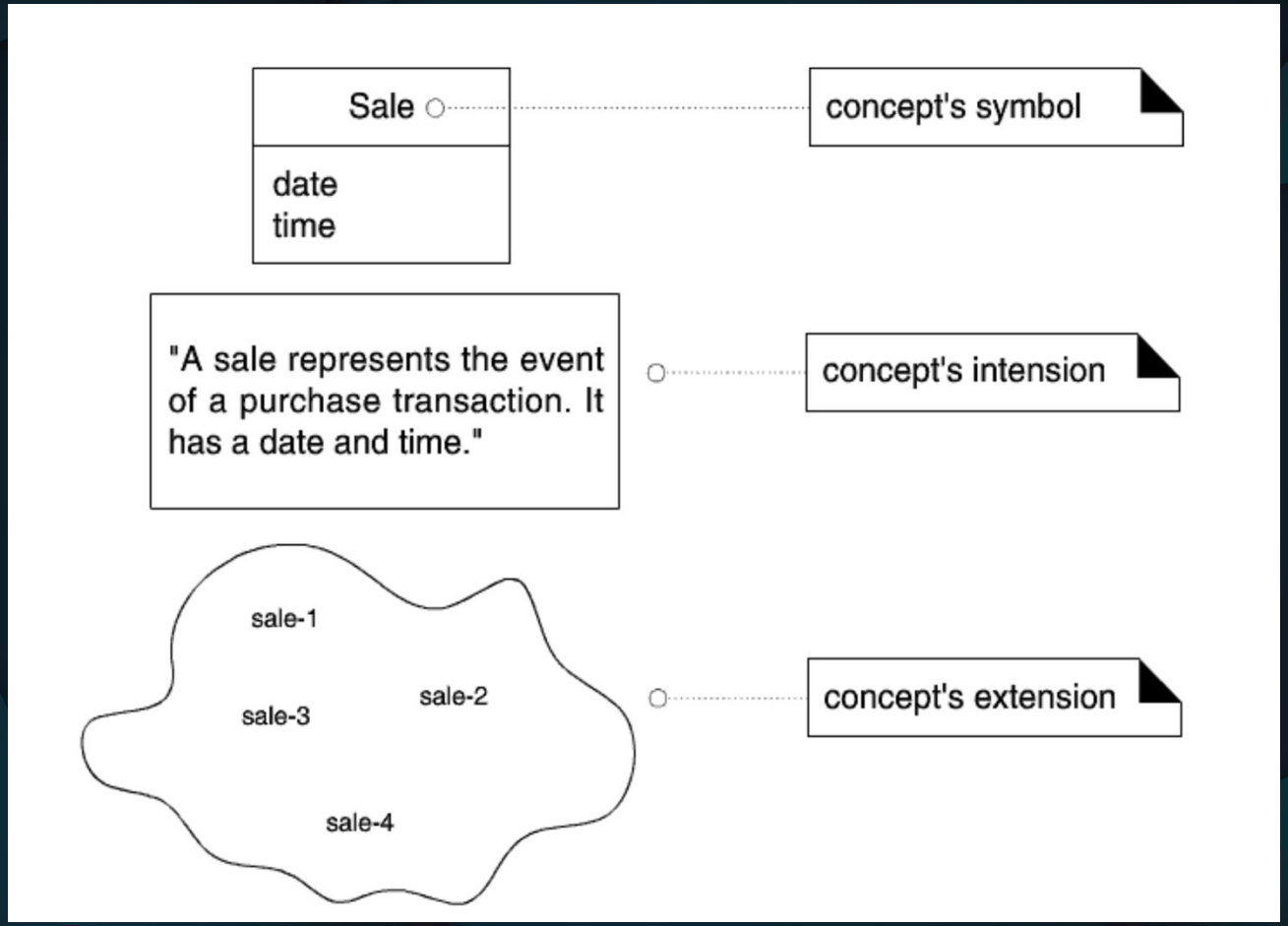
Class names – capital letter

Attribute names - lowercase

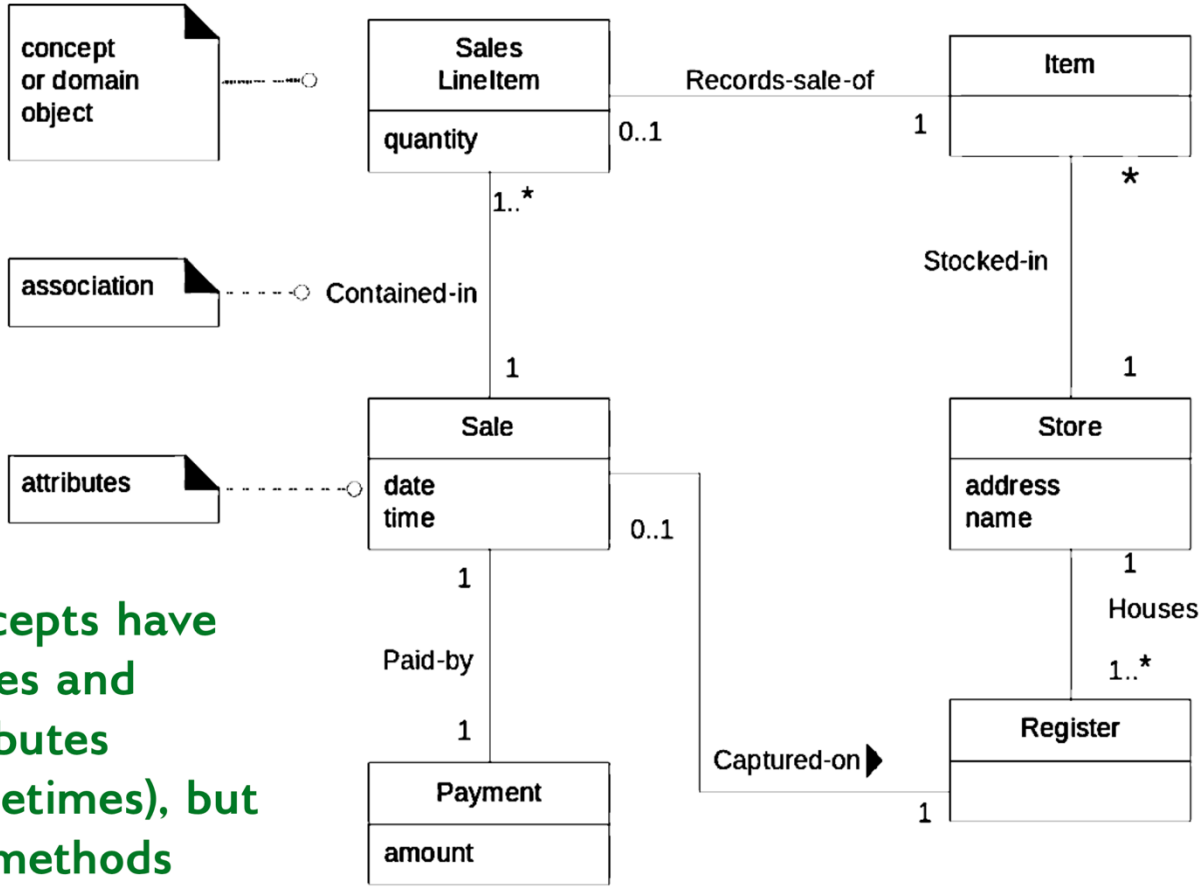




The Domain Model



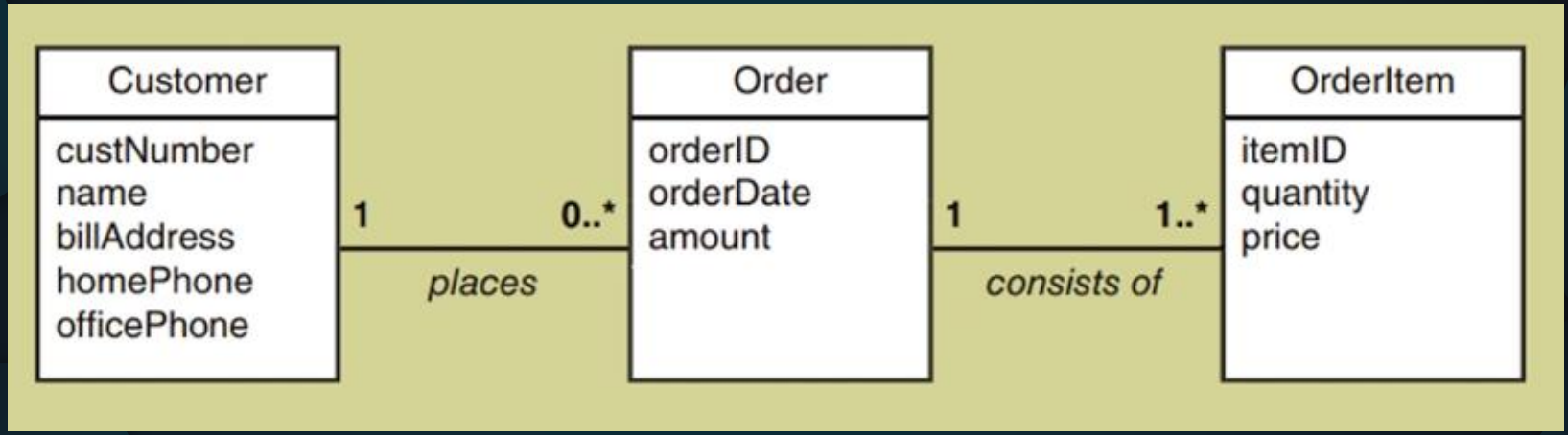
The Domain Model



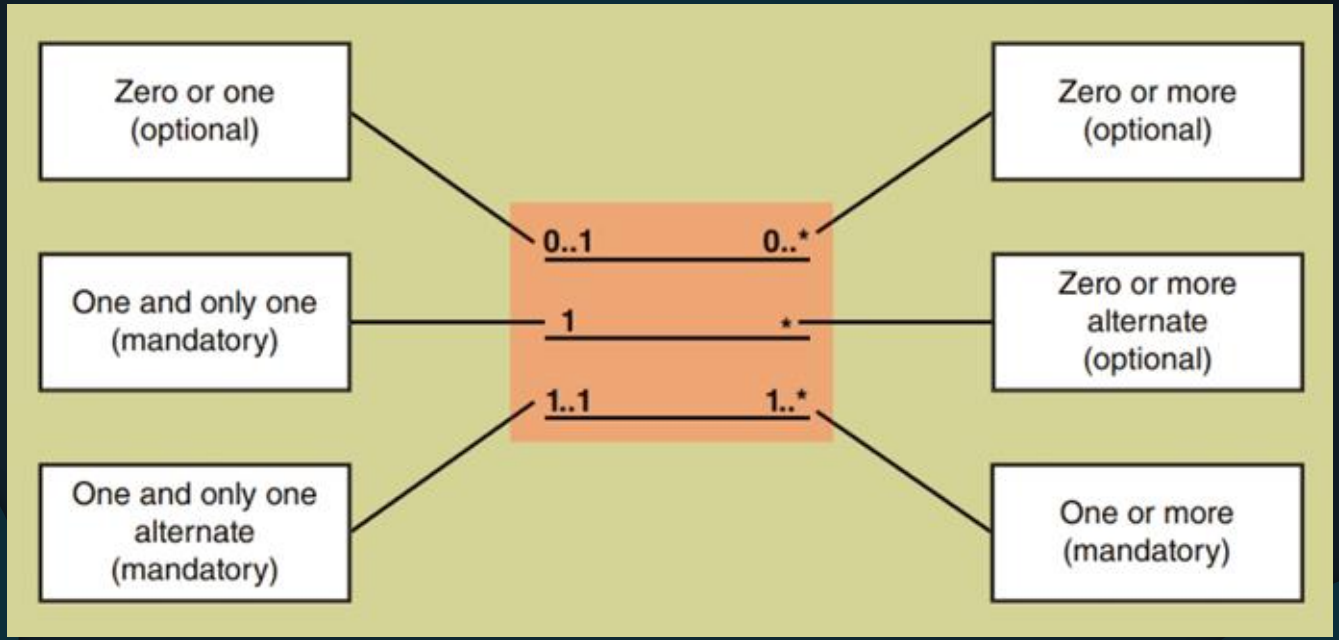
Concepts have names and attributes (sometimes), but not methods

Multiplicity constraints

Example

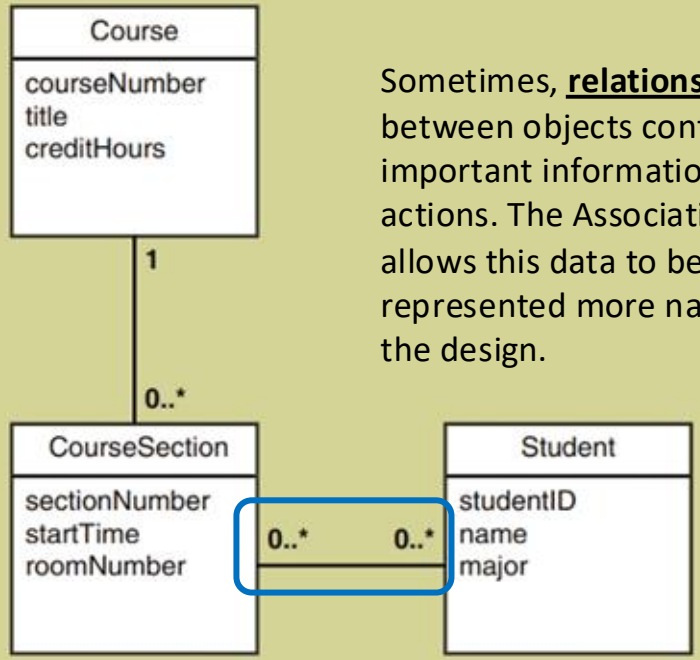


Multiplicity of associations



An association class

Example

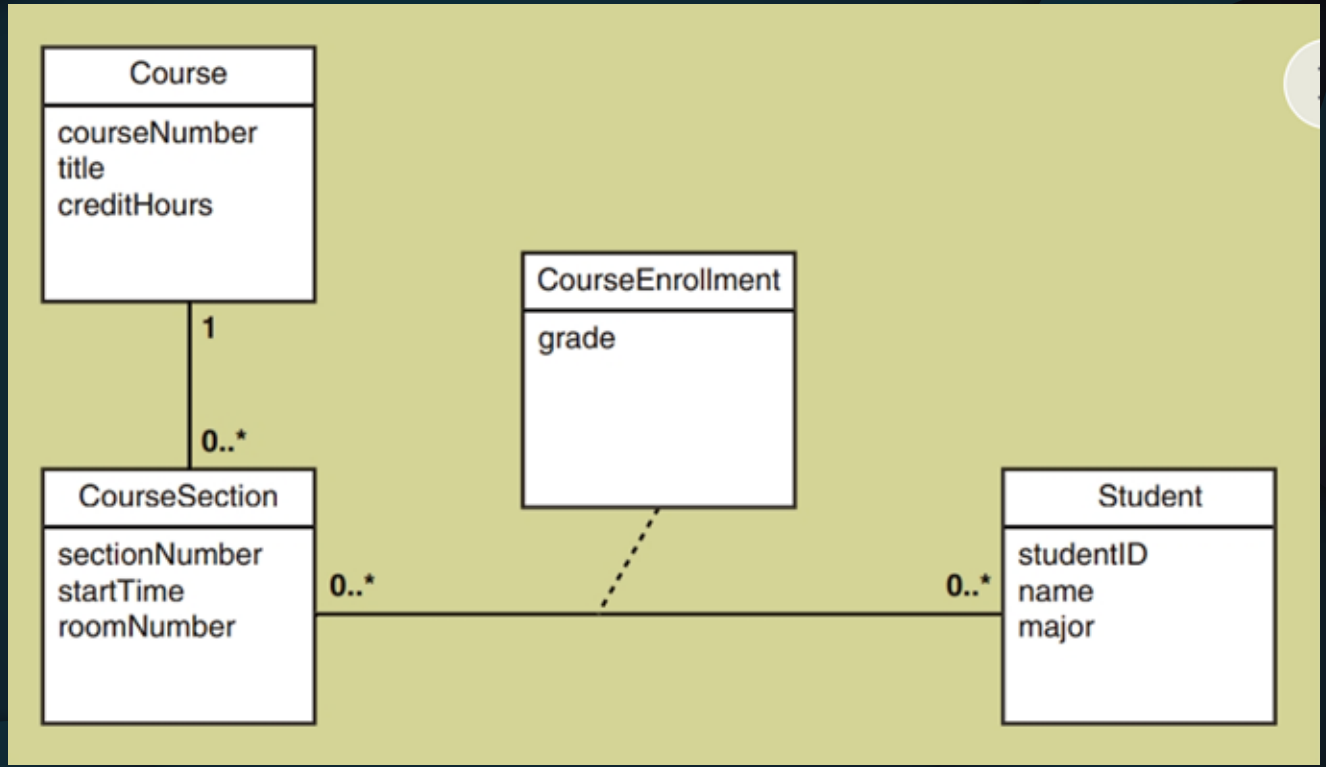


Sometimes, **relationships** between objects contain important information or actions. The Association class allows this data to be represented more naturally in the design.

```
[Student] ------------------ [Enrollment] ------------------ [Course]
                          (enrollmentDate, grade, semester)
```

An association class

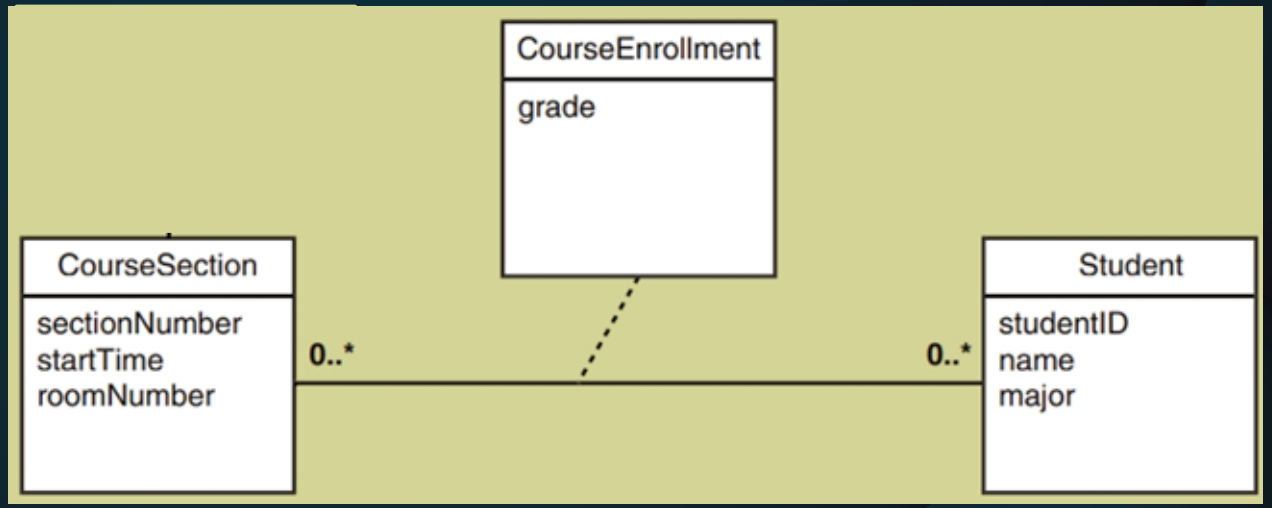
An association class is an association that is also treated as a class; often required in order to capture attributes for the *many-to-many* association.



An association class

one **course section** has **many course enrollments**, each with its own grade, and each **course enrollment** applies **to one specific student**.

one **student** has **many course enrollments**, each with its own grade, and each **course enrollment** applies **to one specific course section**.



An association class

Suppose you have:

- Student: **Anna**
- Course: **Database Systems**
- Course: **Machine Learning**
- Course: **Statistics**

What would happen if you put grade in Student?

Anna can have three different grades:

Student	Course	Grade
Anna	Database Systems	A
Anna	Machine Learning	B
Anna	Statistics	A

Student	Student
studentID name grade	studentID name grades = {A, B, A}

Why this is problematic?

An association class

Suppose you have:

- Course: **Database Systems**
- Course: **Machine Learning**
- Course: **Statistics**

What would happen if you put grade in Course?

Anna can have three different grades:

Student	Course	Grade
Anna	Database Systems	A
John	Database Systems	C
Lena	Database Systems	A

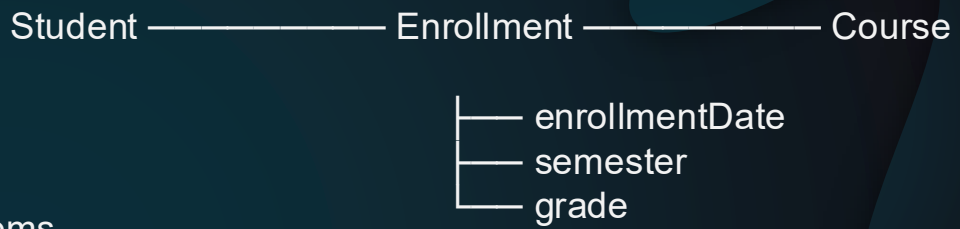
Course
courseID
name
grade

Course
course code
title
credits
description
department

Why this is problematic?

An association class

The key idea: the grade belongs to the **relationship**



Anna ——— Enrollment ——— Database Systems

- semester: Fall 2026
- date: 15 Aug 2026
- grade: A

John ——— Enrollment ——— Database Systems

- semester: Spring 2026
- date: 15 Apr 2026
- grade: C

The **same course** can therefore have different grades because the grade belongs to each **Student–Course association**

An association class

Imagine Anna fails Database Systems

Anna

- Enrollment #1
  - Course = Database Systems
  - Semester = Fall 2025
  - Grade = F
- Enrollment #2
  - Course = Database Systems
  - Semester = Fall 2026
  - Grade = A

Can this attribute exist meaningfully if I know only the Student?

Can this attribute exist meaningfully if I know only the Course?

Do I need to know BOTH the Student and Course to determine this value?

Student attributes describe the student.
Course attributes describe the course.
Enrollment attributes describe what happened when a particular student took a particular course.

Generalization/specialization relationships

…are a type of hierarchical relationship in which subordinate classes are subsets of objects of the superior classes; an inheritance hierarchy.

- based on the idea that people classify things in terms of similarities and differences.

Generalizations are judgments that group *similar types* of things.

Specializations are judgments that group *different types* of things.

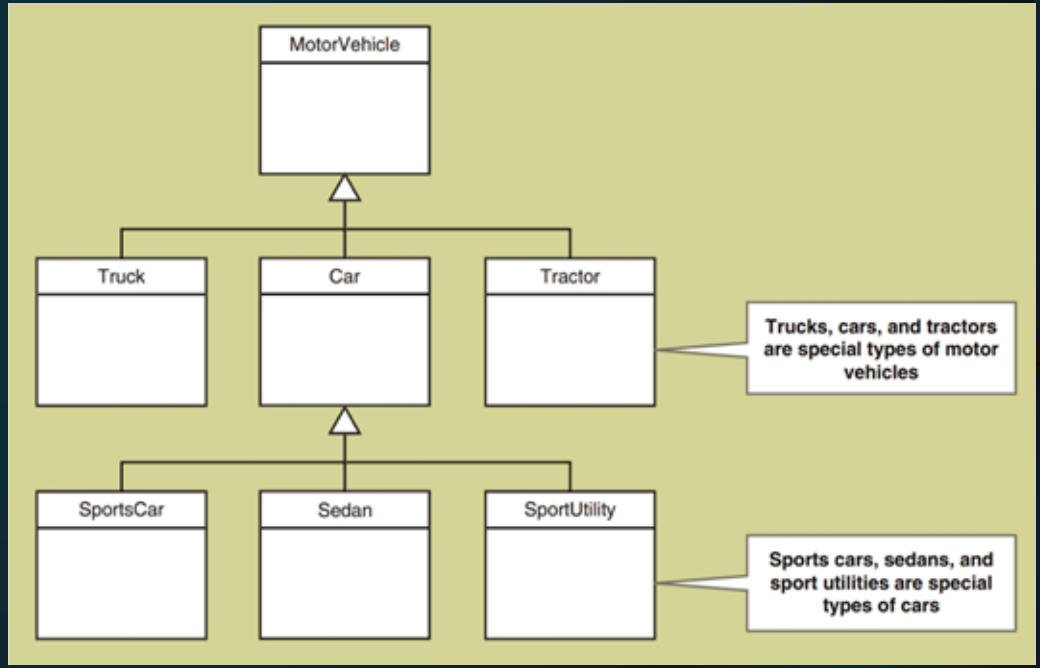
A generalization/specialization relationship is used to structure or rank things from the more general to the more special.

Generalization/specialization relationships

Superclass is a superior or more general class in a generalization/specialization relationship.

Subclass is the subordinate or more specialized class in a generalization/specialization relationship.

Each class of things in the hierarchy might have a more general class above it, called a superclass. At the same time, a class might have a more specialized class below it, called a subclass.



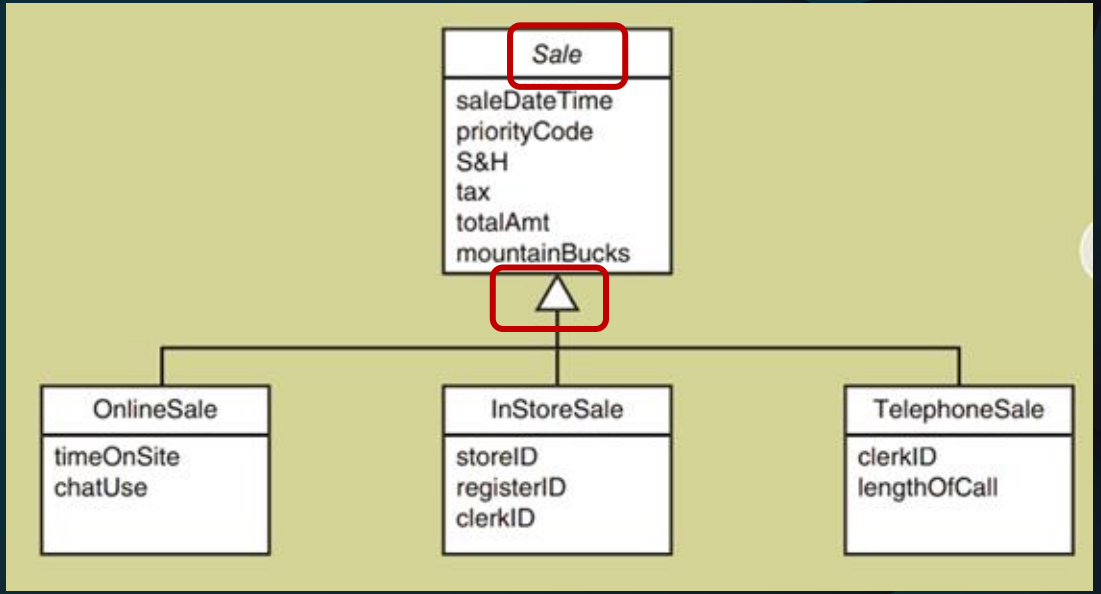
Inheritance

In this example, is there an actual object called sale?

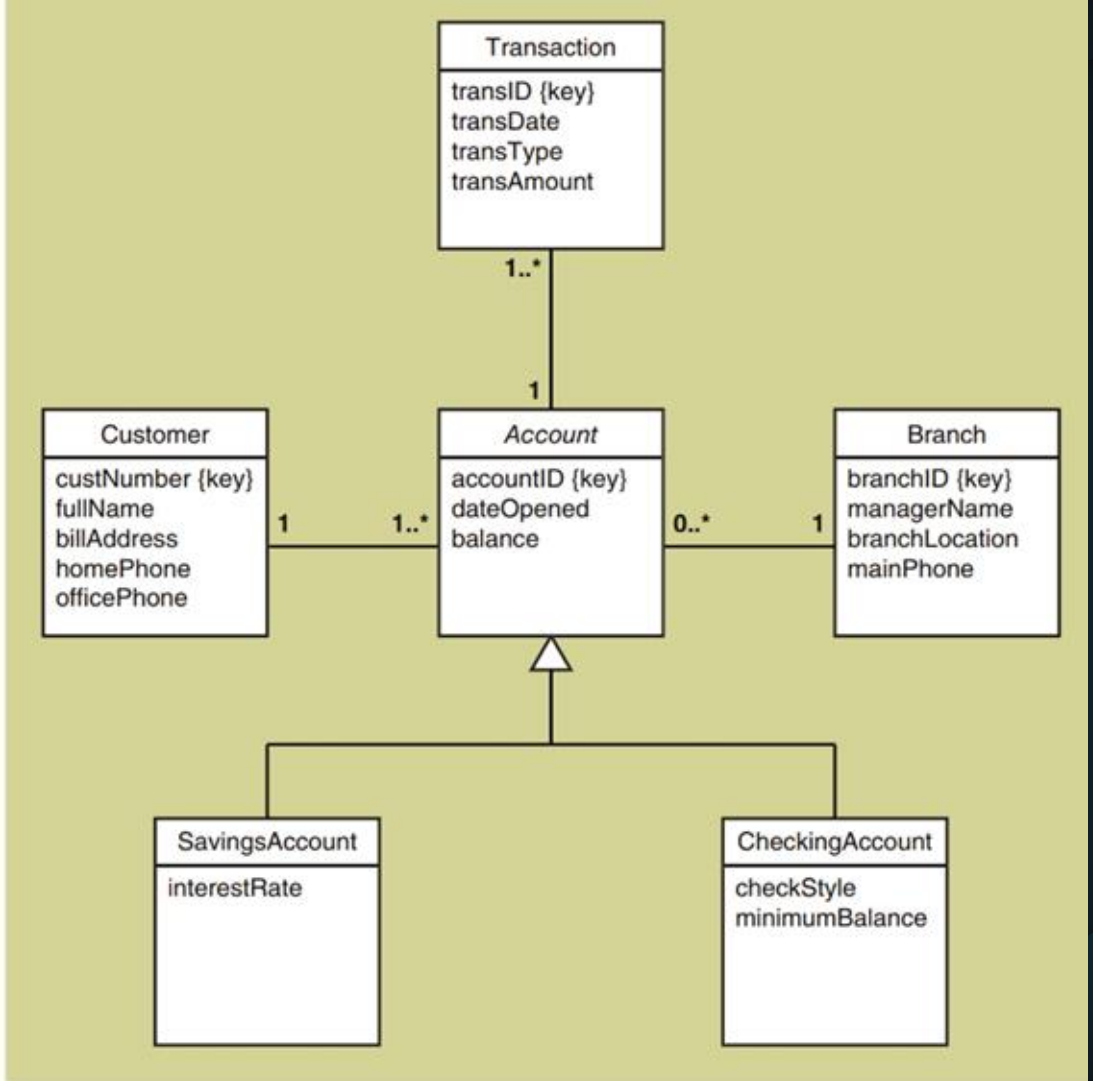
An **abstract class** is a class that exists so subclasses can inherit from it. It describes a category or set of objects but that never includes individual objects or instances.

*Abstract class can not be instantiated.

An abstract class provide some common functionality across a set of related classes while also allowing default method implementations.



Example


Transaction
transID {key}
transDate
transType
transAmount
1..*
1
Customer
custNumber {key}
fullName
billAddress
homePhone
officePhone
1
1..*
Account
accountID {key}
dateOpened
balance
0..*
1
Branch
branchID {key}
managerName
branchLocation
mainPhone
SavingsAccount
interestRate
CheckingAccount
checkStyle
minimumBalance

Whole-part relationships

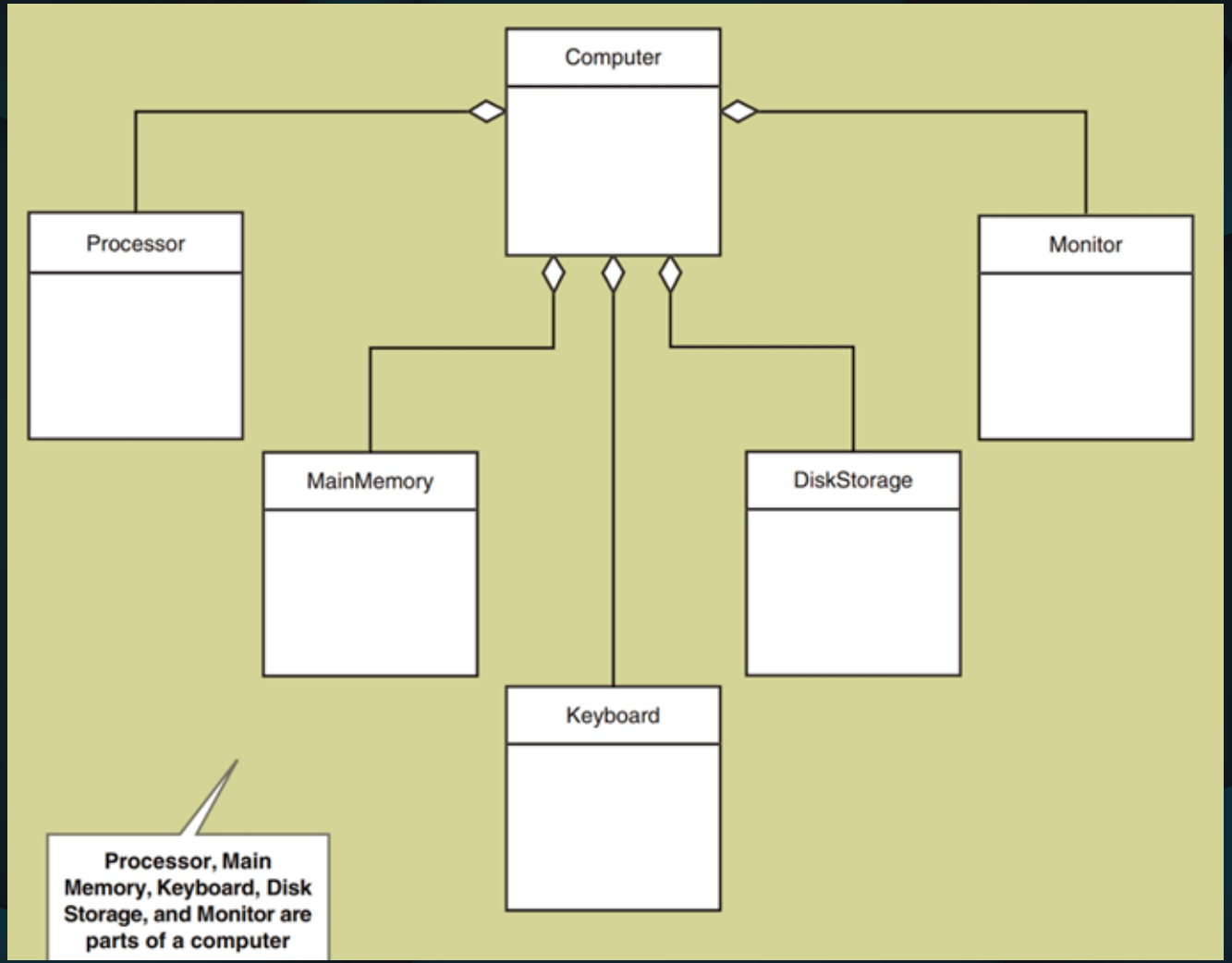
…are a type of a relationship between classes in which one class is a part or a component portion of another class.

- based on the idea that people structure information in terms of their parts.

Aggregation refers to a type of whole-part relationship in which the component parts also exist as individual objects apart from the aggregate.

Composition refers to whole-part relationships that are even stronger, where the parts, once associated, can no longer exist separately.

Example


Computer
Processor
Monitor
MainMemory
DiskStorage
Keyboard
Processor, Main Memory, Keyboard, Disk Storage, and Monitor are parts of a computer



Inheritance: Implements or Extends "Is a"



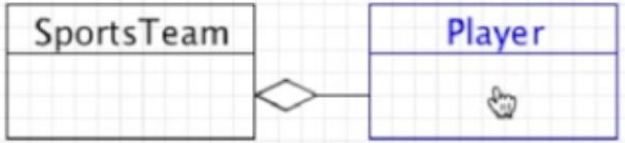
Dependence: Class depends on something, but it isn't a member of the class "Uses a"



Association: Class contains a reference to another class "Has a"

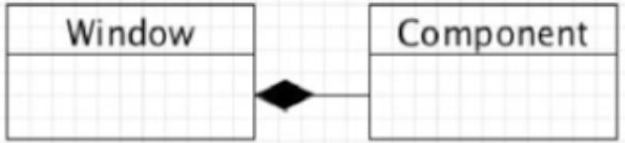


Aggregation: Class is a container for other classes, but if the container is destroyed the contained is not "Owns a"





Composition: Class is a container for other classes and if the container is destroyed the contained is also "Part of"



Does anyone have any questions?

THANK YOU!

katerina.mangaroska@usn.no